Evaluación parcial número 2

MesaTech Cloud con mensajería y persistencia independiente

DSY1107 · Desarrollo Cloud Native I

Trabajo grupal de 2 a 3 integrantes · Continuidad de la evaluación parcial 1

Propósito

Evolucionar MesaTech Cloud mediante dos microservicios Spring Boot desplegados en instancias EC2 diferentes, cada uno conectado a su propia instancia Amazon RDS, e integrados mediante RabbitMQ instalado desde cero con Docker Compose. El equipo deberá demostrar publicación, consumo y persistencia de datos en ambos microservicios.

Contexto y desafío

MesaTech necesita que el servicio de Solicitudes disponga de las categorías de atención mantenidas por el servicio de Catálogo, sin consultar directamente la base de datos de otro servicio. Cuando un administrador cree una categoría, Catálogo deberá guardarla en su RDS y publicar un evento. Solicitudes consumirá ese evento y almacenará una copia de la categoría en su propio RDS para utilizarla al registrar solicitudes.

La copia puede actualizarse con un pequeño retraso: el procesamiento es asíncrono. La interfaz deberá permitir reconocer cuándo la categoría ya está disponible para crear una solicitud. No es necesario que una respuesta del publicador espere a que el consumidor termine.

Continuidad con la EP1

- Se conserva el caso de soporte tecnológico, sus actores y las responsabilidades de Solicitudes y Catálogo.
- Se mantienen React con MSAL, Microsoft Entra ID, AWS API Gateway, el BFF con validación y autorización JWT, las reglas de estados y la coexistencia de v1 y v2.
- React continúa accediendo a las funciones de negocio únicamente por API Gateway. El BFF no accede directamente a RDS ni reemplaza al publicador o consumidor.
- Se reemplaza la alternativa de base de datos en EC2 por Amazon RDS obligatorio. Dos esquemas dentro de una misma instancia RDS no cumplen la separación solicitada.
- Se reemplaza la posibilidad de alojar ambos microservicios en una EC2 por dos instancias EC2 diferentes.

Organización

El equipo entregará código, informe de evidencias y presentación. Todos los integrantes deben comprender el flujo completo. La fecha, duración de la presentación, ponderación en la asignatura y conversión a nota serán informadas por el docente. La pauta de este documento utiliza 100 puntos.

1 Arquitectura y despliegue obligatorios

Distribución de componentes

Componente	Ubicación	Responsabilidad
Catálogo y publicador	EC2 A	Mantener categorías y prioridades; guardar categorías y publicar eventos.
Solicitudes y consumidor	EC2 B diferente de A	Mantener solicitudes y sus estados; consumir categorías y guardar la copia local.
RabbitMQ	EC2 C diferente de A y B	Broker instalado desde cero mediante compose.yaml.
RDS Catálogo	Instancia RDS exclusiva	Categorías y prioridades originales del catálogo.
RDS Solicitudes	Otra instancia RDS exclusiva	Solicitudes y copia de categorías recibidas.
BFF	EC2	Mantener seguridad y coordinación de la EP1; puede compartir A o B.

Se requieren al menos tres EC2 y dos instancias RDS. No se exige una cuarta EC2 para el BFF. Cada microservicio utiliza exclusivamente las credenciales y el endpoint de su propio RDS. No se permite sustituir el intercambio de eventos por lecturas SQL cruzadas, archivos compartidos o una llamada HTTP entre publicador y consumidor.

Flujo que se debe implementar

React → API Gateway → BFF → Catálogo → RDS Catálogo.

Catálogo → exchange de RabbitMQ → cola → consumidor de Solicitudes → RDS Solicitudes.

React → API Gateway → BFF → Solicitudes → consulta de categorías locales y registro de solicitudes en RDS Solicitudes.

Requisitos de las EC2 y las bases de datos

- Identificar cada EC2 por su instance ID, nombre, IP privada y proceso desplegado. Puertos distintos en una sola EC2 no demuestran separación.
- Utilizar Ubuntu 24.04 LTS en la EC2 del broker, siguiendo el laboratorio. Documentar versiones de Java, Spring Boot, Docker y Compose utilizadas. No se impone una familia o tamaño específico de EC2.
- Crear y configurar las dos instancias RDS, sus bases, usuarios de aplicación, conectividad y tablas. Se sugiere MySQL por continuidad con el laboratorio; otro motor RDS es válido si se documentan driver y puerto.
- Presentar identificadores RDS y endpoints distintos, junto con configuraciones de conexión sin contraseñas y consultas a las tablas de ambas instancias.
- Si ya existían recursos RDS de la EP1, entregar evidencia verificable de su creación y adaptación. La instalación nueva de RabbitMQ sí debe demostrarse desde una EC2 sin broker preconfigurado.
- Mantener datos después de reiniciar las aplicaciones. Externalizar credenciales y excluir del repositorio los archivos con valores reales.

2 Instalación de RabbitMQ y conectividad

Instalación desde cero

El equipo debe instalar Docker Engine y el plugin Docker Compose en la EC2 C, construir su archivo compose.yaml y ejecutar la instalación. No se acepta como única evidencia conectarse a un broker preparado por otro equipo, usar Amazon MQ ni instalar RabbitMQ directamente en el sistema operativo.

El compose.yaml debe declarar la imagen oficial de RabbitMQ con una etiqueta explícita que incluya la interfaz de administración, un hostname estable, credenciales administradas fuera del repositorio, publicación de puertos 5672 y 15672, volumen persistente para /var/lib/rabbitmq y una política de reinicio. Dockerizar RabbitMQ no exige construir una imagen propia ni dockerizar los microservicios.

Entregar evidencia de Docker y Compose instalados; validación con docker compose config --quiet; descarga con docker compose pull; arranque con docker compose up -d; estado con docker compose ps; logs y diagnóstico del broker. Las capturas no deben mostrar secretos. Registrar los comandos ejecutados y explicar el propósito de cada configuración relevante.

Recursos de mensajería

Configurar un virtual host del proyecto, usuarios diferenciados para publicador y consumidor, permisos sobre ese virtual host, un exchange direct durable, una cola durable sin autoeliminación y su binding. Los nombres deben corresponder al dominio MesaTech; por ejemplo, mesatech, catalogo.exchange, solicitudes.catalogo.queue y categoria.creada.

Se admite configurar estos recursos mediante la interfaz de administración, código o un archivo de definiciones. La entrega debe permitir reproducirlos. No usar guest como usuario remoto de las aplicaciones. Explicar los permisos otorgados y evitar asignar privilegios administrativos a los usuarios de aplicación.

Matriz mínima de comunicaciones

Destino	Puerto TCP	Origen autorizado y uso
EC2 RabbitMQ	5672	EC2 de Catálogo y Solicitudes mediante sus security groups; conexión AMQP por red privada.
EC2 RabbitMQ	15672	IP administrativa autorizada con máscara /32; interfaz de gestión.
Cada RDS	3306 en MySQL	Security group de la EC2 de su microservicio propietario. Adaptar el puerto si cambia el motor.
EC2 administradas	22 si se usa SSH	IP administrativa autorizada con máscara /32.
APIs y BFF	Puerto configurado	Documentar origen y propósito según la integración conservada de la EP1.

Mostrar el mapeo de puertos del contenedor y las reglas de AWS: ambos son necesarios. Documentar también cualquier firewall del sistema operativo. No habilitar acceso mundial a RDS, AMQP o administración. Mantener RDS sin acceso público y demostrar las consultas desde la EC2 correspondiente o mediante un túnel autorizado. No se exige una topología de red avanzada.

3 Publicación y consumo con persistencia

Contrato mínimo del evento

Al crear una categoría válida, Catálogo debe guardar el registro en RDS y publicar un JSON con eventId, tipoEvento, fechaEvento, categoriaId y nombre. El identificador categoriaId permite relacionar el registro original con la copia recibida; no debe reemplazarse por una clave autogenerada sin conservar la referencia de origen. Documentar tipos, formato de fecha y un ejemplo real del mensaje.

Para el alcance mínimo basta el evento de creación. Las actualizaciones y bajas del catálogo mantienen el comportamiento de la EP1; propagar también esas operaciones por eventos es una ampliación opcional. La prueba evaluada utilizará categorías nuevas, sin modificarlas durante su ejecución.

Responsabilidad del publicador

- Validar la solicitud, persistir la categoría y publicar en el exchange con la routing key acordada.
- Publicar mensajes persistentes y comprobar la confirmación del broker y los retornos por ausencia de una ruta a la cola, siguiendo el laboratorio.
- Si no se confirma la publicación, entregar una respuesta de error controlada e indicar que el registro pudo quedar guardado. No afirmar que todo se revirtió automáticamente.
- Conservar en las evidencias la relación entre la solicitud HTTP, categoriaId, eventId o messageId, y el mensaje publicado.

Responsabilidad del consumidor

- Escuchar la cola desde Solicitudes en la EC2 B, convertir el JSON y guardar la categoría recibida en una tabla de su propio RDS.
- Confirmar el procesamiento solo después de guardar correctamente. Se admite el modo AUTO de Spring AMQP del laboratorio si el listener retorna después de finalizar la persistencia y propaga los errores.
- Permitir consultar las categorías locales mediante un endpoint protegido por el flujo de la EP1 y utilizarlas al registrar una solicitud.
- Mostrar logs sin secretos que permitan relacionar el evento con el registro persistido. No leer el RDS de Catálogo para demostrar el consumo.

Persistencia y límites del flujo

La confirmación del publicador indica aceptación por RabbitMQ; no acredita la escritura del consumidor en RDS. Una cola durable conserva su definición, mientras que la recuperación de mensajes requiere además publicación persistente y almacenamiento conservado del broker. La prueba de reinicio debe mantener el volumen.

El guardado en RDS y la publicación son operaciones separadas. El equipo debe explicar qué ocurre si la primera tiene éxito y la segunda falla, y qué riesgo existe al reenviar una solicitud o volver a entregar un mensaje. No se exige implementar outbox, transacciones distribuidas, idempotencia, DLQ ni reintentos avanzados; no debe afirmarse una garantía de procesamiento exactamente una vez.

4 Pruebas y evidencias obligatorias

Registrar para cada prueba la preparación, datos utilizados, resultado esperado, resultado obtenido y evidencia identificada. Las pruebas P2 a P5 deben permitir seguir una misma categoría o un conjunto claramente identificado de categorías.

Prueba	Ejecución y resultado esperado
P1 Separación real	Mostrar IDs diferentes de EC2 A y B y dos identificadores y endpoints RDS distintos. Relacionar cada proceso con su conexión y tabla.
P2 Flujo completo	Crear una categoría desde React con usuario administrador. Verificar el registro en RDS Catálogo, publicación, log del consumidor y fila en RDS Solicitudes. Crear una solicitud usando la categoría recibida.
P3 Consumidor detenido	Detener el consumidor y crear otra categoría. La categoría queda en RDS Catálogo y el mensaje aparece Ready en la cola; todavía no existe en la copia local.
P4 Consumo diferido	Iniciar el consumidor de P3. Comprobar que procesa el mensaje pendiente, lo guarda en su RDS y termina sin mensajes Ready o Unacked para ese caso. La cola vacía por sí sola no demuestra persistencia.
P5 Reinicio del broker	Con consumidor detenido, publicar otra categoría y verificar que hay un mensaje pendiente. Recrear el contenedor conservando volumen y hostname, sin borrar volúmenes. Comprobar que siguen el virtual host, exchange, cola y mensaje; iniciar el consumidor y verificar la escritura.
P6 Reinicio de aplicaciones	Reiniciar los dos microservicios y volver a consultar datos previamente guardados. No deben perderse los registros de ninguno de los RDS.
P7 Fallo de publicación	Detener temporalmente RabbitMQ e intentar crear otra categoría. Mostrar respuesta controlada y consultar el RDS del publicador para documentar el resultado real. Restablecer el broker y explicar la posible inconsistencia, sin exigir recuperación automática.
P8 Continuidad de EP1	Probar login y consumo autorizado; rechazo sin token y por permisos insuficientes; regla que impide resolver sin EN_PROCESO; disponibilidad de v1 y v2.

Evidencias de configuración

Además de las pruebas, incluir capturas de creación e instalación de RabbitMQ, compose.yaml sin secretos, contenedor activo, recursos del virtual host, permisos de usuarios, conexiones, security groups y configuración de ambos RDS. Identificar qué evidencia respalda cada requisito y evitar capturas aisladas sin explicación.

No usar Get messages con confirmación que retire el mensaje durante las pruebas de espera y reinicio. Si se inspecciona el contenido desde la interfaz, utilizar una opción que lo devuelva a la cola y documentarla.

5 Entrega y presentación

Código y documentación reproducible

- Repositorio o archivo con React, BFF y los dos microservicios, identificando la versión o commit entregado.
- compose.yaml de RabbitMQ, .env.example con valores ficticios y documentación para crear virtual host, usuarios, permisos, exchange, cola y binding.
- README con versiones, requisitos, compilación, despliegue, configuración de variables, arranque, detención y orden de recuperación de los componentes.
- Contrato del evento, endpoints utilizados y scripts SQL o mecanismo documentado para crear las tablas. No entregar contraseñas, claves privadas, tokens ni archivos .env reales.

Informe técnico

El informe debe explicar los cambios respecto de la EP1, presentar el diagrama real de despliegue, inventariar EC2 y RDS, describir puertos y orígenes autorizados y documentar la instalación de RabbitMQ desde cero. Debe incluir P1 a P8 y una reflexión breve sobre publicación, confirmaciones, persistencia y fallos.

El diagrama debe mostrar React, Entra ID, API Gateway, BFF, las tres EC2, ambos RDS y las flechas del flujo de mensajes y de acceso a datos. Indicar explícitamente qué componente publica, cuál consume y quién es propietario de cada base de datos.

Presentación y defensa individual

La presentación seguirá la modalidad de la EP1: resumen de configuraciones y capturas de funcionamiento, sin una demostración en vivo obligatoria durante la exposición. El docente podrá solicitar levantar los servicios y repetir alguna prueba para verificar las evidencias.

Cada integrante responderá preguntas sobre la separación de EC2 y RDS, compose.yaml, puertos 5672 y 15672, exchange, cola, binding, confirmaciones y resultados de las pruebas. La defensa individual corresponde a 10 de los 100 puntos de la pauta; los otros 90 corresponden al trabajo grupal.

Fuera de alcance

No se exige agregar microservicios, rehacer la interfaz, dockerizar las aplicaciones Java, usar Amazon MQ, Lambda, ECS, EKS, Kubernetes, CI/CD, infraestructura como código, alta disponibilidad o servicios de observabilidad avanzada. Tampoco se exige implementar outbox, DLQ, idempotencia o una solución de transacciones distribuidas.

Lista de comprobación

- Los dos microservicios están en EC2 diferentes y cada uno utiliza una instancia RDS exclusiva.
- RabbitMQ fue instalado desde cero en otra EC2 utilizando el compose.yaml entregado.
- Las reglas de puertos, los usuarios y el virtual host permiten el flujo requerido.
- Las evidencias muestran datos en ambos RDS y mensajes pendientes y consumidos.
- El volumen de RabbitMQ y los datos de RDS sobreviven a las pruebas indicadas.
- Se conserva el acceso seguro y el comportamiento mínimo de MesaTech Cloud.

6 Pauta de evaluación

Los criterios C1 a C8 suman 90 puntos grupales. C9 suma 10 puntos individuales. En cada criterio se asigna el porcentaje del máximo según el nivel alcanzado: completo 100 %, mayoritario 75 %, parcial 50 %, inicial 25 % o ausente 0 %.

Completo significa cumplir y demostrar todos los elementos indicados. Mayoritario significa cumplir el resultado principal con una omisión menor de documentación. Parcial significa demostrar solo parte del resultado o presentar una falla relevante. Inicial significa presentar configuración o código sin acreditar el resultado. Ausente significa no presentar evidencia evaluable. Registrar una justificación y la evidencia utilizada para cada calificación.

Criterio	Máximo	Resultado para obtener el puntaje completo
C1 Despliegue EC2	10	Catálogo y Solicitudes operativos en EC2 distintas; broker en tercera EC2; IDs y diagrama verificables.
C2 Persistencia RDS	20	Dos instancias exclusivas, conexiones propias y escritura y consulta en ambas; datos conservados al reiniciar aplicaciones.
C3 Docker Compose	15	Instalación nueva de Docker y RabbitMQ, compose.yaml reproducible, imagen explícita, puertos, volumen, hostname y política de reinicio; diagnóstico satisfactorio.
C4 Red y acceso	10	Reglas de origen y puerto justificadas para AMQP, gestión, RDS y administración; conectividad operativa y credenciales externalizadas.
C5 Mensajería	10	Virtual host, usuarios, permisos, exchange, cola y binding correctos; publicación persistente, confirmaciones y consumo tras persistir.
C6 Integración y pruebas	15	P2 a P7 trazables: flujo, espera, recuperación y fallos con resultados comprobables en RabbitMQ y ambos RDS.
C7 Continuidad EP1	5	P8 satisfactoria y categoría recibida utilizable al crear solicitudes desde el flujo seguro de React.
C8 Entregables	5	Código, README, contrato, informe y presentación completos, coherentes y reproducibles.
C9 Defensa individual	10	Explica el recorrido del dato, configuración de infraestructura y diferencia entre confirmaciones; interpreta un fallo con evidencia del proyecto.
Total	100	90 puntos grupales más 10 individuales.

Reglas para aplicar la pauta

Si ambos microservicios comparten EC2, C1 tiene un máximo de 5 puntos. Si comparten una instancia RDS, aunque tengan esquemas distintos, C2 tiene un máximo de 10 puntos; sin RDS, C2 obtiene 0. Si RabbitMQ no se despliega mediante compose.yaml, C3 obtiene 0; si funciona con Compose pero no se demuestra instalación desde cero, C3 tiene un máximo de 7,5 puntos.

Aplicar primero el nivel de logro y después el límite correspondiente. No descontar puntos adicionales fuera de la pauta por el mismo incumplimiento. Otros criterios se evalúan por sus evidencias propias. Estas reglas no establecen una nota mínima ni una condición automática de aprobación.

7 Referencias técnicas

Utilizar las guías de clase para adaptar el patrón publicador y consumidor al dominio MesaTech. Verificar compatibilidad de las versiones utilizadas y explicar las configuraciones propias; copiar el ejemplo de ventas sin adaptarlo al caso no satisface el objetivo.

Material de la asignatura

- EP1_Caso_Negocio_MesaTech_Cloud (2).pdf: arquitectura, negocio, seguridad y entregables que se conservan.
- Paso a Paso (ventas-publicador).docx: persistencia, publicación de JSON, confirmaciones y consumo con escritura en otra base de datos.
- instalacion nuevo servidor RabbitMQ.docx: preparación de EC2 Ubuntu, instalación de Docker y Compose, arranque y puertos.

Documentación oficial

- Docker Engine en Ubuntu: https://docs.docker.com/engine/install/ubuntu/
- Servicios de Docker Compose, puertos y volúmenes: https://docs.docker.com/reference/compose-file/services/
- Puertos y conectividad de RabbitMQ: https://www.rabbitmq.com/docs/networking
- Confirmaciones de publicador y consumidor: https://www.rabbitmq.com/docs/confirms
- Persistencia de RabbitMQ: https://www.rabbitmq.com/docs/persistence-conf
- Conectividad de RDS mediante security groups: https://docs.aws.amazon.com/AmazonRDS/latest/gettingstartedguide/security-groups.html

Los puertos 5672 y 15672 corresponden a AMQP y administración HTTP sin TLS, respectivamente. En este laboratorio, AMQP se utiliza por red privada y la administración se restringe al origen autorizado. La publicación de puertos de Docker y las reglas de security groups son controles distintos que deben verificarse conjuntamente.

La prueba de reinicio evalúa conservación local de los datos del broker, no alta disponibilidad ni recuperación ante pérdida de la EC2 o de su almacenamiento.